计算机网络面试题速记通关版 _ 面试刷题 mianshiya.com

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*

TCP 中何时会出现 RST（reset）报文？

RST 报文是 TCP 连接中的一种强制终止信号，表示当前连接出现了异常或不可恢复的状态，直接“掐断”连接。

1）当客户端向一个**没有监听的端口**发起连接请求时，服务端会回一个 RST。比如你往 8080 端口发 SYN，但那边根本没开服务，内核协议栈就会直接回 RST，告诉对方“这地方没人，别连了”。

2）连接已经关闭后，如果还有**残留数据包**发过来，接收方会回 RST。典型场景是连接被 close() 了，但对端还懵着继续 send，收到后内核发现这个连接在本地已经没了，就直接 RST 告辞。

3）TCP 协议栈检测到**序列号不合法**也会触发 RST。比如你发了个 ACK 包，但 seq 不在当前连接的接收窗口内，系统判断这是个错包或伪造包，干脆 RST 断掉，防止状态混乱。

4）程序崩溃或进程被杀，操作系统回收 socket 时，可能来不及发 FIN，对端还在发数据，这时候也会由内核代发 RST。和正常四次挥手不同，RST 是硬断，**压根不经过握手流程**。

代码上没法主动控制 RST 发送，它是内核协议栈的行为。但你可以通过 `SO_LINGER` 设置为 0 来让 close() 时直接发 RST 而不是等待 FIN，用于快速释放连接。

这种机制保证了 TCP 的健壮性，避免无效连接堆积。但频繁出现 RST 通常意味着架构设计或网络配置有问题，比如负载均衡转发到了未就绪实例，或者客户端重试逻辑太激进。

常见的登录鉴权方式有哪些？各自的优缺点是？

登录鉴权这事，说白了就是确认“你是你”的过程。现在主流的几种方式，基本都围绕着怎么安全又高效地维持这个身份状态。

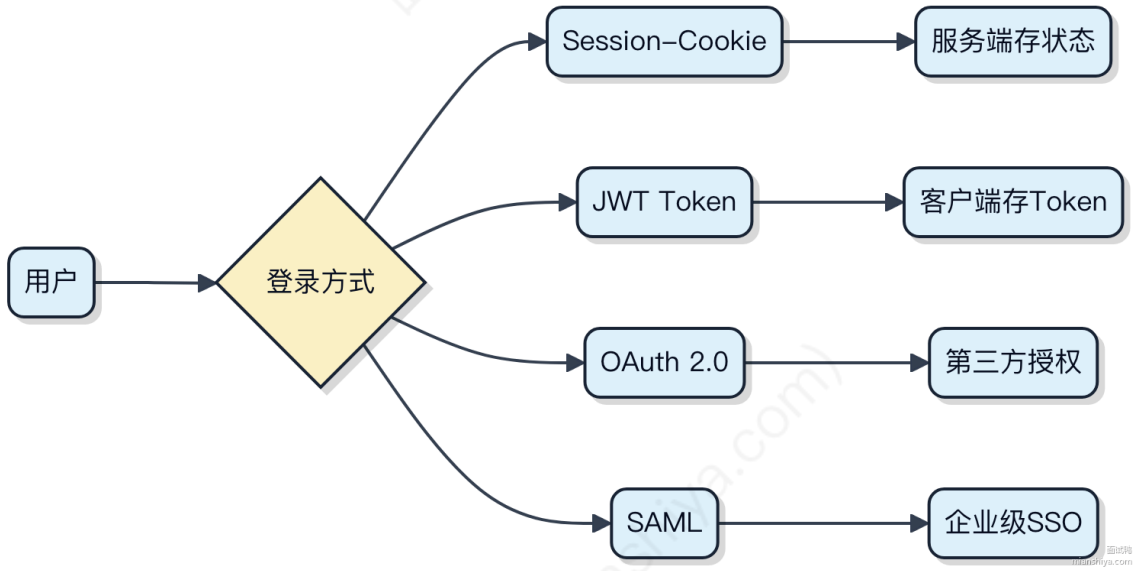
1）Session-Cookie
用户登录后，服务端生成一个 Session 存在内存或 Redis 里，把 sessionId 写进 Cookie 返回浏览器。后续请求自动带

上 Cookie，服务端靠它查状态。
优点是逻辑清晰，支持主动登出。但问题是服务端要存状态，横向扩容时得搞 Session 共享，比如用 Redis 集中存储，否则负载均衡到不同机器就认不出来了。

2）Token（如 JWT）
登录成功后，服务端把用户信息签成一个 Token 发回去，客户端自己存，每次请求放 Header 带上。服务端收到后通过密钥验签，不依赖本地存储。
好处是无状态，适合分布式系统，像 Spring Cloud、Kubernetes 这种架构下很轻便。但 Token 一旦签发，无法中途撤销，除非加黑名单机制，反而变重了。而且别把敏感信息直接塞进 Payload。

3）OAuth 2.0 / OpenID Connect
不是直接做登录，而是授权框架。比如用微信登录第三方网站，你不用给密码，微信只返回一个 access token。
OpenID Connect 在 OAuth 基础上加了 identity 层，真正实现“登录”。
适合开放平台和单点登录（SSO），像钉钉、GitHub 登录都能这么搞。但流程复杂，一般小项目没必要。

4）SAML
老一点的企业级 SSO 方案，基于 XML，配置繁琐，但现在很多传统公司还在用，比如和 AD FS 对接。



TCP/IP 四层模型是什么？

TCP/IP 不是死记硬背的四层，而是为了解决网络通信问题演化出来的分层设计。每一层各司其职，把复杂的联网拆解成可管理的部分。

1）链路层
负责物理传输，比如以太网、Wi-Fi 这一层处理 MAC 地址和帧的收发。交换机工作在这里，数据包压根不经过 IP，直接靠 MAC 转发。

2）网络层
核心是 **IP 协议**，负责主机到主机的寻址和路由。路由器根据 IP 地址转发数据包，中间可能经过十几个节点。ICMP、ARP 也归这一层管。

3）传输层
提供端到端的通信能力。TCP 保证可靠、有序、重传，适合 HTTP、数据库连接；UDP 不保证送达，但快，音视频通话、DNS 查询就用它。

4）应用层
所有协议都为具体应用服务。HTTP、HTTPS、FTP、SMTP 都属于这层。你的程序代码最常接触的就是这一层接口。

实际抓包时你会发现，一个 HTTP 请求背后是四层协议层层封装：应用数据加上 TCP 头，再加 IP 头，最后套上以太网帧头，发出去。

说说 TCP 拥塞控制的步骤？

TCP 拥塞控制不是靠一个算法搞定的，是四个机制组合起来动态调整发送速率，让网络别崩。

刚开始发包的时候，不知道网络啥情况，就从慢启动开始。每收到一个 ACK，拥塞窗口 cwnd 就加 1，指数增长。比如发 1 个，确认了变成 2 个，再确认变 4 个……一直到达到慢启动阈值 ssthresh。

1）慢启动阶段：cwnd < ssthresh，指数增

一旦 cwnd 超过 ssthresh，就进拥塞避免阶段。这时候不再指数涨，而是每轮只加 1，线性增长。比如这一轮发了 10 个包，全确认了，下一轮发 11 个。

2）拥塞避免阶段：cwnd >= ssthresh，线性增

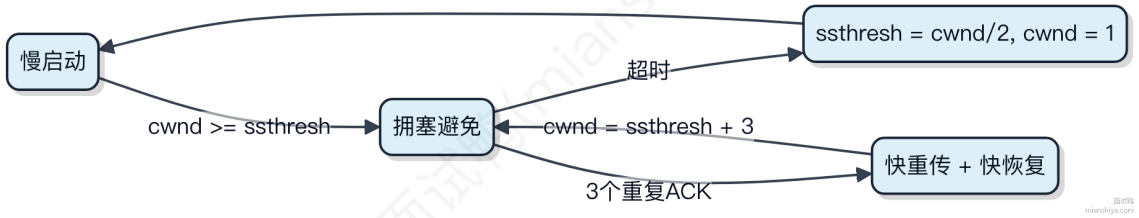
如果发生超时重传，说明可能严重拥塞了。这时候把 ssthresh 设为当前 cwnd 的一半，然后 cwnd 重置为 1，重新慢启动。

3）超时处理：ssthresh = cwnd / 2，cwnd = 1，回慢启动

如果是收到三个重复 ACK，触发快重传，不用等超时。接收方连续发了三次同样的 ACK，发送方立马重发丢的那个包。接着执行快恢复，把 ssthresh 减半，但 cwnd 设为新 ssthresh + 3，直接进拥塞避免，不用回零。

4）快恢复：ssthresh = cwnd / 2，cwnd = ssthresh + 3，进拥塞避免

整个过程就是试探着往上加，碰到问题就降一点，再稳着走，保证吞吐同时不压垮网络。



JWT Token 能说说吗？

JWT 本质就是把用户信息编码成一个字符串，让服务端不用存 session 也能验证身份。你发一次登录请求，服务器生成这个 token 返回给你，之后每次请求都带上它。

它由三部分组成，用点号分隔：头部、载荷、签名。头部说明算法，载荷放业务数据比如用户 ID 和过期时间，签名是用 secret 密钥对前两部分加密生成的，防篡改。

```
// 生成示例，用 jjwt 库
Jwts.builder()
    .setSubject("12345") // 用户ID
    .setExpiration(new Date(System.currentTimeMillis() + 86400000))
    .signWith(SignatureAlgorithm.HS512, "your-secret-key")
    .compact();
```

客户端一般放在 Authorization 头里传过来，格式是 `Bearer <token>` 。服务端拿到后用同样的密钥校验签名，解析出用户信息，整个过程**无状态**，适合分布式系统。

但要注意几个问题。一是 token 一旦签发，在有效期内始终有效，没法像 session 那样主动销毁，想控制就得加黑名单机制，比如用 Redis 记录失效列表。二是别在载荷里放敏感信息，因为载荷是 Base64 编码，能解出来。

适用于前后端分离、微服务间认证，比如 Spring Cloud Gateway 做统一鉴权时常用。但高安全场景要谨慎，毕竟拿住 token 就能冒用。

简单谈谈你对 DNS 的理解？

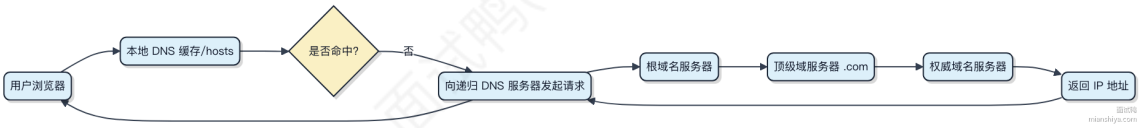
DNS 本质上就是把域名翻译成 IP 地址的一个分布式系统。你访问 google.com，最终得知道它背后是哪个 IP 的服务器在扛着，这个“翻译”工作就是 DNS 干的。

整个过程不是一蹴而就的，一般会经历几个环节。本地查 hosts 文件，没命中就问本地配置的 DNS 服务器，比如运营商给的或者你手动设的 8.8.8.8。这台服务器如果缓存里有就直接返回，没有就得一层层往上问：先问根域名服务器，再问顶级域（比如 .com），最后找权威域名服务器拿到结果。

1）递归查询：客户端只管发请求，期望拿到最终答案，不管中间谁去跑腿。
2）迭代查询：DNS 服务器之间互相问，每次只返回下一级该找谁，自己不继续往下追。

缓存在这套体系里特别关键，几乎每一层都会缓存一段时间（TTL 控制），减少重复查询压力，也提升响应速度。但 TTL 没过期前，改了记录可能不会立刻生效。

举个例子，用 Nginx 或者 CDN 做负载均衡时，通常会给一个域名配多个 A 记录，DNS 返回的时候可以轮询，实现最基础的流量分发。不过这种方案粒度粗，健康检查弱，真要搞高可用还得靠 LVS 或者云厂商的负载服务。



简单谈谈你对 CDN 的理解？

CDN 的本质是把内容搬到离用户更近的地方，减少访问延迟和源站压力。你打开一个网站，静态资源比如图片、JS、CSS 可能不是从原服务器拿的，而是从几十公里外的边缘节点拉取，这就是 CDN 在起作用。

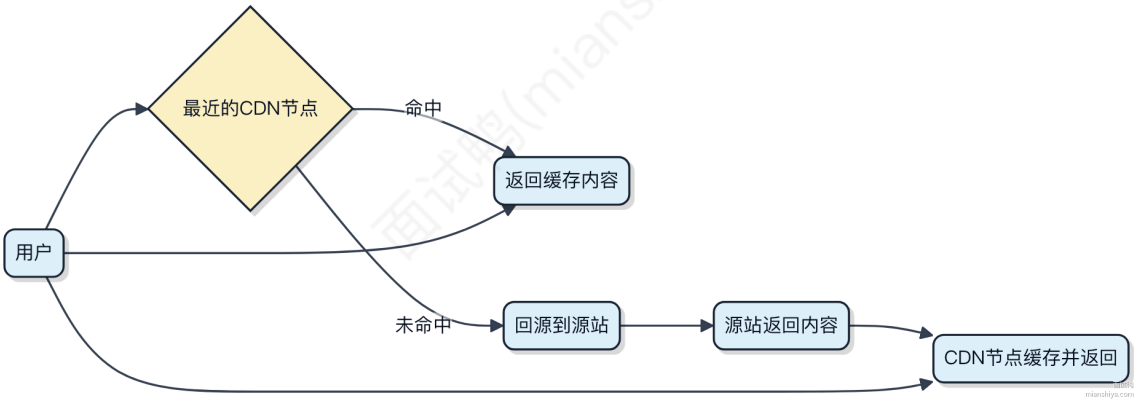
1）CDN 节点分布在全球各地，用户请求时通过 DNS 调度，自动接入最近的节点。如果节点缓存有数据，直接返回，响应时间从几百毫秒降到几十毫秒。

2）缓存策略很关键，一般通过 URL 和 HTTP 头（如 Cache-Control）决定是否命中。比如 `Cache-Control: max-age=3600` 表示一小时内不回源。

3）回源是脏活累活，当缓存失效或被穿透，请求才会打到源站。像阿里云 CDN、Cloudflare 都支持配置回源协议、Host 头、甚至压缩优化。

4）动态加速也有玩法，比如腾讯云的 DSA、网宿的动态加速技术，通过链路优化、TCP 优化把动态请求也“加速”。

CDN 不是万能的，缓存一致性是个问题，更新了资源可能全网要等 TTL 过期。所以通常配合版本号或哈希文件名使用，比如 `app.abc123.js` ，一变就换新 URL。



别指望 CDN 解决所有性能问题，它主要扛住静态流量的大头，尤其是突发流量，比如秒杀活动前把商品页静态资源推到 CDN，能帮你省下一大笔带宽成本。

TCP 超时重传机制是为了解决什么问题？

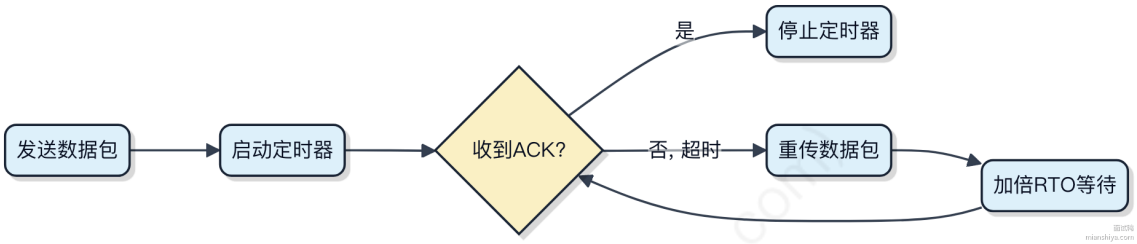
网络传输不是理想环境，数据包可能在路上丢了，或者回执丢了。TCP 作为可靠的传输协议，必须能发现这种情况并补救，不然应用层就收不到完整数据。

超时重传解决的就是**丢包**问题。发送方发出去一个报文段后，会启动一个定时器。如果在规定时间内没收到对方的 ACK 确认，就认为这个包丢了，于是重新发一次。整个过程对上层应用透明。

这个“规定时间”叫 RTO（Retransmission Timeout），它不是固定值。TCP 会根据当前网络的实际往返时间 RTT 动态计算 RTO。比如刚开始估计是 200ms，后来发现网络变慢了，RTT 升到 500ms，RTO 也会跟着调整，避免误判重传。

1）每次发数据都会启动定时器
2）ACK 到了就清除对应定时器
3）超时未到 ACK 就重发并加倍等待时间（指数退避）

这样既能应对暂时的网络波动，又不会在网络彻底卡住时疯狂重试。



如果连续重传几次（一般是 15 次）还是没响应，TCP 连接就会被断开。这种机制让 TCP 能在不可靠的 IP 层之上，建立起可靠的数据通道。

常见的 HTTP 状态码有哪些？

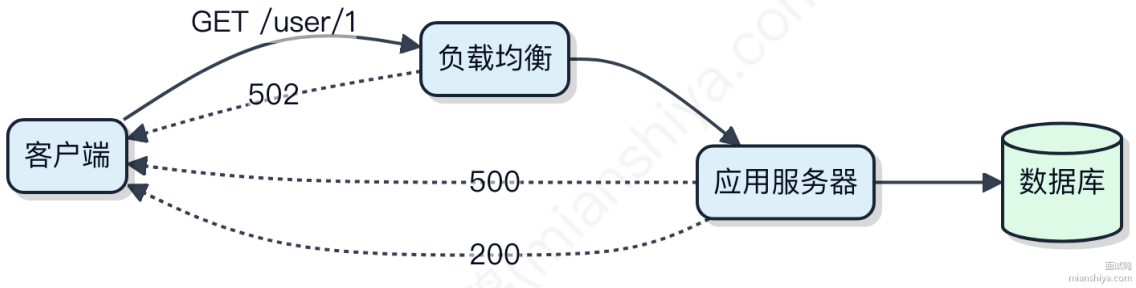
1xx 这类状态码表示临时响应，比如 100 Continue 是客户端发请求时先问一句“我接下来要发数据，你准备好接收了吗”，服务器回 100 就代表可以继续。

200 是最熟的，请求成功，服务端把数据正常返回。201 表示资源创建成功，比如 POST 提交一个用户，数据库真加进去了。204 是成功处理了请求但没内容返回，像 DELETE 操作后常用。

301 和 302 都是重定向。301 是永久搬走，比如老域名彻底不用了，搜索引擎会更新索引；302 是临时跳转，比如登录后跳回原页面。现在还有 307、308，它们保证重定向时请求方法不变，比如 POST 不会变成 GET。

400 是客户端发的数据不对，比如 JSON 格式错了。401 是没登录，需要认证，比如没带 token。403 是登录了但没权限，比如普通用户访问管理员接口。404 大家都懂，路径写错了或者资源不存在。

500 是服务器自己出问题了，代码抛异常没处理。502 是网关错误，比如 Nginx 转发请求给下游服务，结果那个服务挂了。503 是服务暂时不可用，可能是过载或维护，常见于 Spring Cloud 或 Kubernetes 场景下实例宕机。



记住几个高频的就行：200、400、401、403、404、500，基本覆盖日常开发 90% 的场景。其他按需查文档也来得及。

说说 TCP 的四次挥手？

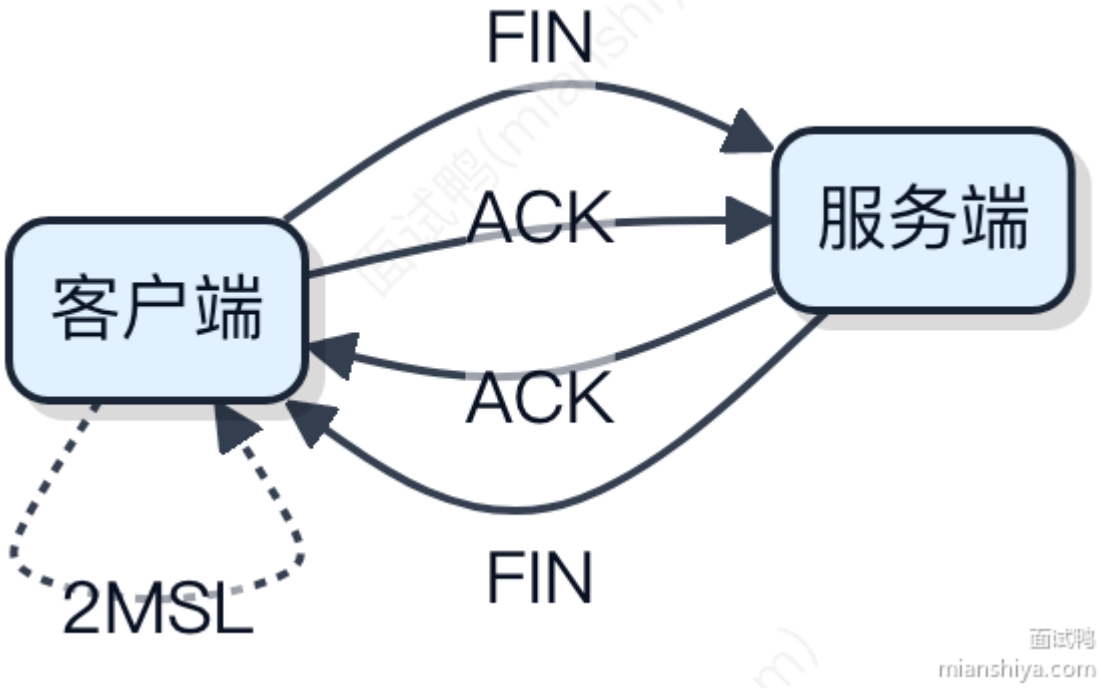
TCP 四次挥手是连接释放的标准流程，确保双方都能可靠地关闭数据传输。

客户端发送 FIN 报文，表示自己不再发数据了。服务端收到后回复 ACK，这个阶段叫半关闭，服务端还能继续发数据。

等服务端也准备好关闭时，会发自己的 FIN 报文。客户端收到后回复最后一个 ACK，等待 2MSL 时间后彻底关闭连接。

整个过程需要四次交互，主要是因为 TCP 是全双工通信，两端必须独立关闭。如果任意一端不确认对方的 FIN，连接就不能安全释放。

1）客户端发 FIN
2）服务端回 ACK（此时客户端进入 FIN_WAIT_2，服务端进入 CLOSE_WAIT）
3）服务端发 FIN
4）客户端回 ACK（进入 TIME_WAIT，服务端收到后连接关闭）



TIME_WAIT 状态很关键，它防止最后一个 ACK 丢包导致服务端重传 FIN 失败。客户端要等够 2 倍的最大报文生存时间，确保网络中残留的该连接报文都消失。

常见问题是在高并发短连接场景下，客户端大量处于 TIME_WAIT 状态，占用端口资源。可以通过开启 `SO_REUSEADDR` 或调整内核参数来缓解。

[HTTP 请求包含哪些内容，请求头和请求体有哪些类型？]

HTTP 请求本质是客户端发给服务端的一段文本，结构上分四块：请求行、请求头、空行、请求体。请求行包含方法、路径和协议版本，比如 `GET /api/user HTTP/1.1` 。

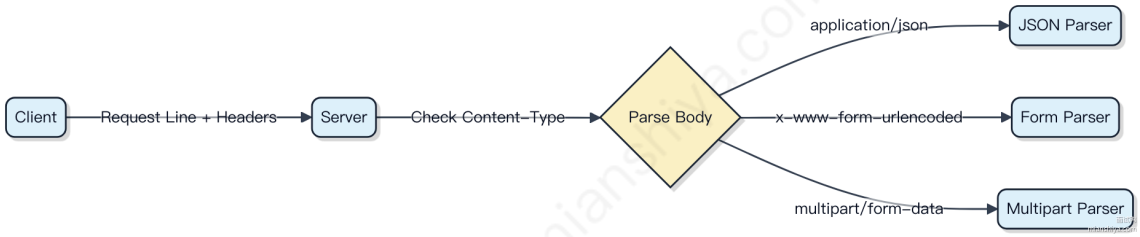
请求头携带元数据，常见类型有： 1） `Content-Type` 告诉服务端请求体的格式，比如发 JSON 用 `application/json` ，传表单用 `application/x-www-form-urlencoded` ，上传文件用 `multipart/form-data` 。2） `Authorization` 放认证信息，像 JWT 就塞在这里。 3） `User-Agent` 标识客户端类型。 4） `Accept` 表示客户端能接收的响应格式，比如 `Accept: application/json` 。 5） `Cookie` 把之前服务端种的 cookie 带回去。

请求体不是每个请求都有，GET、HEAD 就没有。POST、PUT 这类方法会带。常见的体类型和 Content-Type 强相关：

- 提交表单时可能是 `name=jack&age=25` 这种键值对。

- 调用 REST API 通常传 `{"name": "jack"}` 这样的 JSON。
- 上传文件就是二进制流，配合 `multipart/form-data` 分段传输。

反过来说，服务端靠请求头里的 `Content-Type` 决定怎么解析请求体。比如 Spring MVC 用 `@RequestBody` 接收 JSON，底层靠 `HttpMessageConverter` 处理。



说说 TCP 的三次握手？

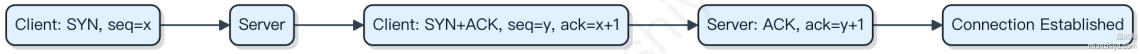
TCP 三次握手是建立连接的必经之路，主要目的是同步双方的初始序列号，确保通信能正常开始。

客户端先发一个 SYN 包，带上自己的初始序列号 seq=x。这个包就像敲门说“我想连你”。服务器收到后，得回应一个 ACK=x+1 表示确认收到，同时再发一个 SYN=y 告诉客户端“我也想连你”，这就是 SYN+ACK 包。最后客户端再回一个 ACK=y+1，表示你也收到了。这时候连接才算真正建立起来。

整个过程涉及的状态变化要清楚。客户端从 SYN_SENT 到 ESTABLISHED，服务器从 LISTEN 进入 SYN_RCVD，最后双方都到 ESTABLISHED。如果中途丢包，比如第一个 SYN 没到，客户端会超时重传，一般重试 5 次，间隔从 1 秒开始指数增长。

为啥不能两次？因为两次无法保证双向通道都通。客户端发了 SYN，服务器回 ACK，但这个 ACK 可能丢失，客户端不知道服务器是否准备好了，直接发数据就可能白发。三次能确保两边都能收也能发。

现实里像 Nginx、Redis 这些服务启动后就在 listen，等着客户端来握手。高并发场景下可能出现半连接队列（SYN 队列）满的情况，这时候新的连接请求会被丢弃，得调大 somaxconn 和 tcp_max_syn_backlog。



最后一次 ACK 如果丢了，服务器会重发 SYN+ACK，客户端收到后会重新回复 ACK，避免连接卡住。

HTTP 中 GET 和 POST 的区别是什么？

GET 和 POST 的本质区别不在“安全性”或者“能传多少数据”，而是**幂等性和语义定义**。

HTTP 方法的设计是基于语义的。GET 的本意是“获取资源”，它应该是安全且幂等的，也就是说，调用一次和调用多次对系统的影响是一样的，不会改变服务器状态。浏览器刷新、预加载这些操作都依赖这个特性。

POST 则是“提交数据”，每次请求可能会创建新资源，比如提交一个订单，连续点两次就可能生成两个订单，所以它不幂等。

很多人说 GET 传参在 URL 里，有长度限制，POST 在 body 里能传大文件，这其实是实现上的表现，不是根本区别。你完全可以在 GET 请求里带 body，只是服务器和中间件一般不处理而已。

1）GET 请求的数据会暴露在 URL 中，会被浏览器历史、服务器日志记录，不适合传敏感信息。
2）传统上 GET 受 URL 长度限制（一般 2048 字符），而 POST 没有理论上限，实际受服务器配置约束。

3）GET 能被缓存，能被收藏为书签；POST 不行。

但关键还是语义。RESTful 接口设计里，获取列表用 GET，创建资源用 POST，就是遵循这种约定。

WebSocket 与 HTTP 有什么区别？

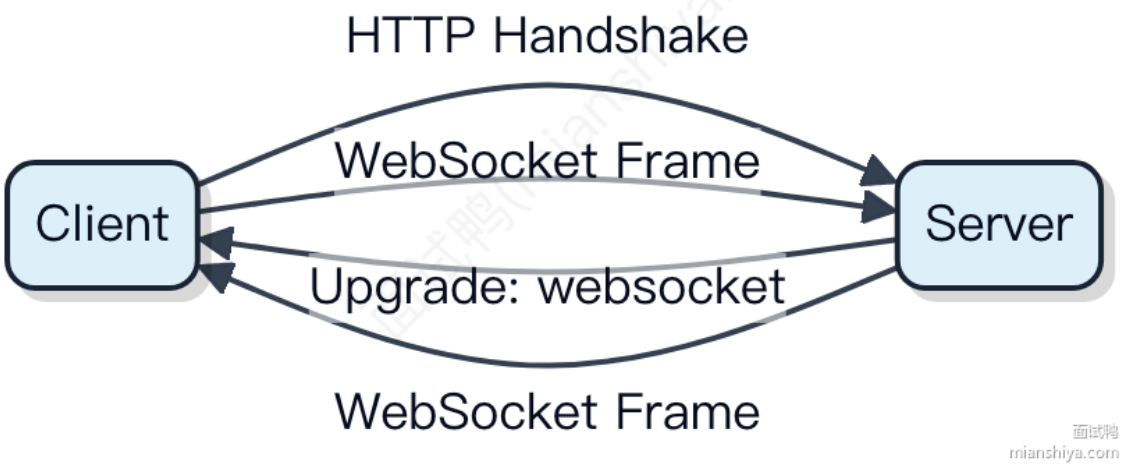
HTTP 是请求-响应模型，一次连接只能发一次请求，拿到响应就断了。WebSocket 不一样，它先靠 HTTP 握手，升级成 **ws** 协议之后，就能在单个 TCP 连接上双向通信。

1）连接方式不同
HTTP 每次都要重新建连接（除非 keep-alive），而且只能客户端发，服务端回。WebSocket 建一次连，双方都能随时推数据，适合聊天、实时行情这种高频交互场景。

2）通信模式
HTTP 是单向的，服务端没法主动通知客户端。WebSocket 是全双工，服务端可以随时往客户端写消息，比如用 Spring WebFlux 或 Netty 实现推送。

3）开销对比
HTTP 头大，每次请求都带一堆 header，浪费流量。WebSocket 握手后帧头很小，传输效率高，尤其对移动端或高并发推送场景更友好。

4）使用场景
HTTP 适合传统网页加载、API 调用这些“问一次答一次”的操作。WebSocket 常见于需要长连接的系统，像钉钉 Web 版聊天、股票 K 线更新、游戏状态同步。



别以为所有实时需求都得上 WebSocket。如果只是偶尔刷新状态，用轮询或者 Server-Sent Events 可能更轻量。真正需要低延迟双向通信时，再上这玩意儿。

TCP 和 UDP 有什么区别？

TCP 和 UDP 的本质区别在于，一个追求可靠交付，一个追求极致效率。

TCP 会建立连接，通过三次握手确认双方收发能力没问题，数据包丢了会重传，顺序乱了会重新排序，保证你收到的数据和发送的一致。像 HTTP、数据库连接这些不能出错的场景都用它，背后是 **可靠性** 在撑腰。

UDP 完全不握手，发出去就完事，不管对方听没听到。没有重传、没有流量控制，但正因为没这么多约束，性能高、延迟低。音视频通话用 UDP 多，比如 WebRTC，偶尔丢个包顶多画面卡一下，总比延迟高强。

1）TCP 有连接状态，维护成本高，并发百万连接对服务器资源消耗大
2）UDP 无状态，每个包独立处理，天生适合做广播、组播，像 DNS 查询、DHCP 分配 IP 都靠它
3）TCP 头部至少 20 字节，UDP 只有 8 字节，头部开销小一半以上

代码上看也很直观：

```
// UDP 发送片段
DatagramSocket socket = new DatagramSocket();
byte[] data = "hello".getBytes();
InetAddress addr = InetAddress.getByName("127.0.0.1");
DatagramPacket packet = new DatagramPacket(data, data.length, addr, 8080);
socket.send(packet); // 扔出去就不管了
```

```
// TCP 发送片段
Socket socket = new Socket("127.0.0.1", 8080);
OutputStream out = socket.getOutputStream();
out.write("hello".getBytes()); // 写完也不一定立刻发，内核会缓冲
```

简单说，要准确就选 TCP，要快就选 UDP。没有谁更好，只有合不合适。

TCP 是用来解决什么问题？

TCP 的本质是解决不可靠网络下的可靠传输问题。现实中的网络会丢包、乱序、重复，甚至中断，而 TCP 就是在这种环境下，让数据像“流水”一样完整、有序地从一端送到另一端。

它不关心你传的是网页、文件还是视频，只保证：你说要发 10000 字节，对方就得收到完整的 10000 字节，少一个都不行。

实现靠的是几个关键机制：

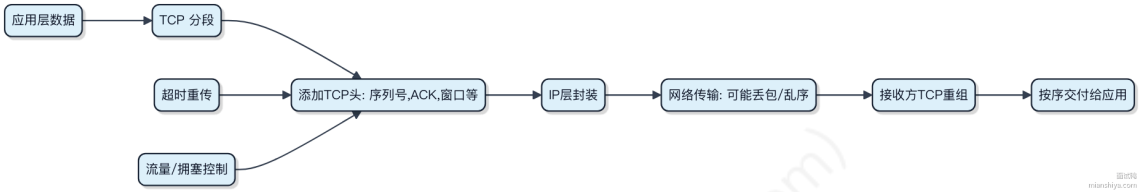
1）序列号和确认应答。每个字节都编了号，接收方回个 ACK 告诉发方“我收到到第多少了”。没收到确认？定时器一到就重发，确保不丢。

2）滑动窗口。不是发一个等一个 ACK，那样太慢。TCP 允许连续发一批，只要在窗口范围内，对方处理得过来就行，效率高。

3）流量控制和拥塞控制。接收方通过窗口字段告诉发送方“我现在只能收这么多”，避免被压垮；网络堵了就主动降速，比如用 Reno 或 Cubic 算法调整发送节奏。

4）连接管理。三次握手建立连接，双方同步初始序列号，避免历史连接干扰；四次挥手确保双方都确认关闭。

典型场景比如 HTTP 协议跑在 TCP 上，浏览器加载页面不能丢包，否则 HTML 解析就错了。而像音视频通话这种实时性要求高、能容忍少量丢失的，则更倾向用 UDP。



所以，TCP 的价值就是为上层屏蔽底层网络的复杂性，提供一条“可靠的字节流管道”。

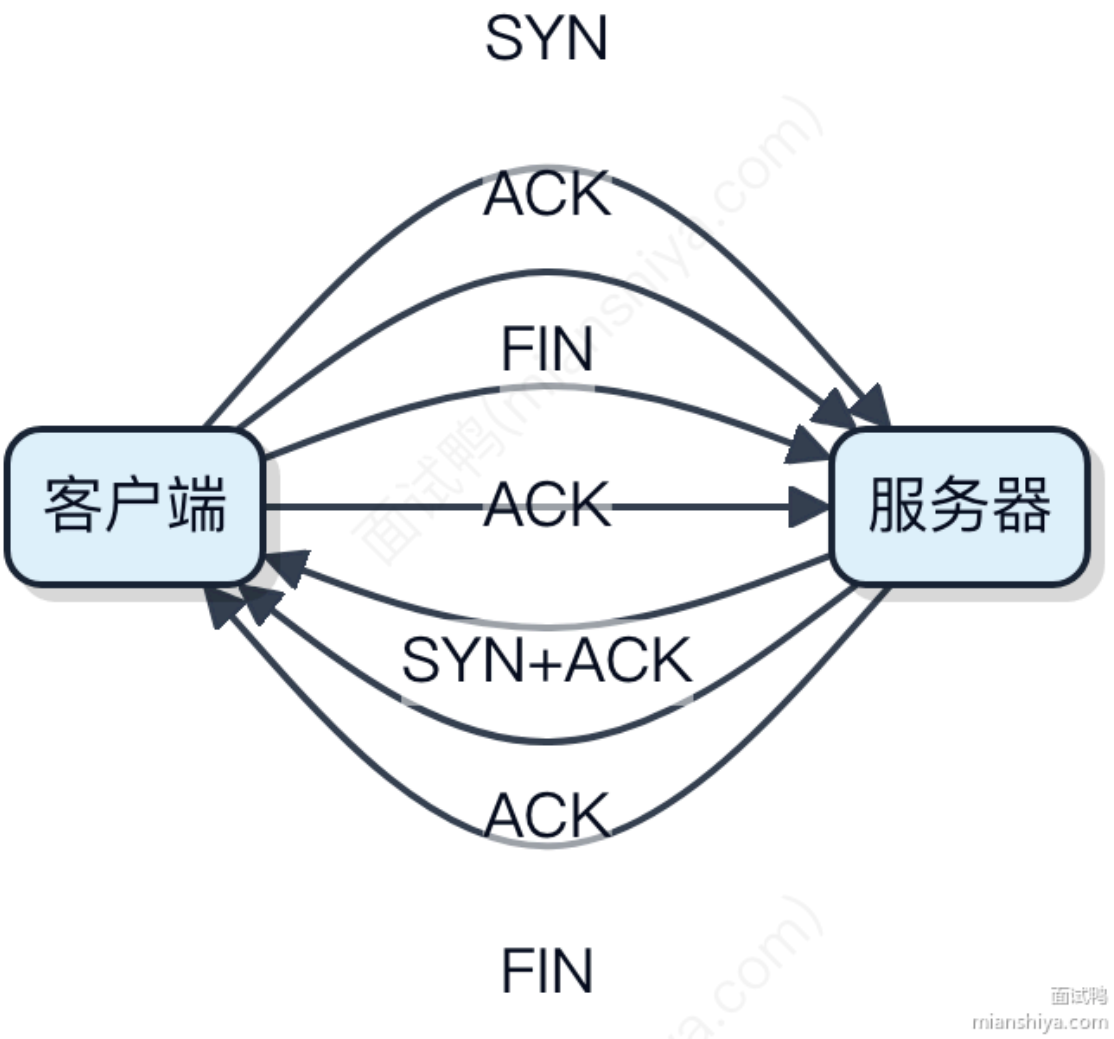
到底什么是 TCP 连接？

TCP 连接本质上是通信双方维护的一对状态，不是物理线路。建立连接靠三次握手，客户端和服务器交换各自的初始序列号（ISN），确认彼此的收发能力。

1）第一次：客户端发 SYN=1, seq=x 给服务器
2）第二次：服务器回 SYN=1, ack=x+1, seq=y
3）第三次：客户端再发 ACK=1, ack=y+1

握手完成后，双方内核里的 TCP 状态机都进入 ESTABLISHED，开始传输数据。断开连接则是四次挥手，因为 TCP 是双工的，读写通道要分别关闭。

面试鸭(mianshiya.com)



很多人以为连接是“通路”，其实它只是操作系统里两个 socket 对象在内核中保存的状态信息，比如缓冲区、窗口大小、序列号等。网络中断或超时，状态不一致，连接就失效了。

像 Nginx、Redis 这类高并发服务，最怕 TIME_WAIT 太多或者文件描述符耗尽，本质都是连接资源没释放。所以得调优内核参数，比如开启 `SO_REUSEADDR` 或调整 `tcp_tw_reuse` 。

状态同步 和 **可靠传输** 才是 TCP 连接的核心目标，它通过序列号、确认应答、超时重传来保证数据不丢、不乱序。连接本身，不过是实现这个目标的上下文容器。

[服务端是如何解析 HTTP 请求的数据？（考察 HTTP 请求格式的了解程度）]

HTTP 请求不是凭空就能处理的，服务端得先把它拆开看明白。整个过程其实分几步走，每一步都对应着 HTTP 协议本身的结构。

请求一进来，TCP 层先把数据收上来，应用层比如 Tomcat 或 Netty 开始按 **HTTP 报文格式**解析。第一行是请求行，包含方法、路径和协议版本，比如 `GET /api/user?id=123 HTTP/1.1` 。这一行决定了你要干啥、去哪。

接下来是请求头，一堆键值对，每一行一个字段。像 `Host` 、 `Content-Type` 、 `Authorization` 都在这儿。服务器靠这些判断怎么处理请求，比如是不是 JSON 数据、有没有登录。

如果请求带了 body，比如 POST 提交表单或 JSON，服务端会根据 `Content-Length` 或 `Transfer-Encoding` 知道 body 有多长。 `Content-Length` 直接告诉你字节数，一口气读完就行。如果是 chunked 编码，就一段段收，直到收到结束标记。

body 的内容还得看 `Content-Type` 。如果是 `application/x-www-form-urlencoded` ，那就按 key=value 解码；如果是 `application/json` ，就得用 Jackson 或 Gson 做反序列化。

整个流程压根不经过业务逻辑，全是容器或框架底层搞定的。你写 Controller 的时候已经拿到解析好的参数了，但底层其实走了一整套状态机在拆包。

TCP 协议是如何保证可靠传输的？

TCP 保证可靠传输不是靠单一机制，而是几个关键设计组合起来的结果。连接建立、数据排序、确认应答、超时重传、流量控制和拥塞控制这几点缺一不可。

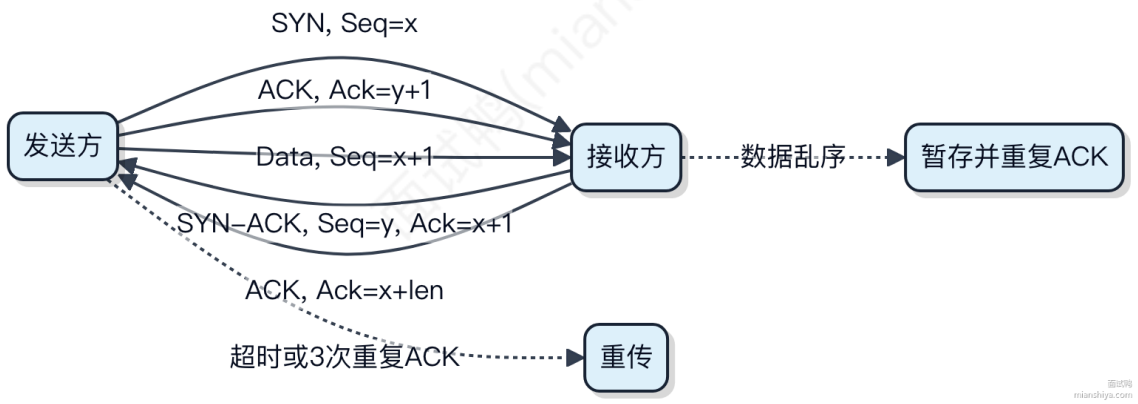
1）三次握手建立连接，确保双方收发能力正常。客户端和服务端各自初始化序列号（ISN），后续数据包都带 Seq 和 Ack 号，实现有序传输和双向确认。

2）每个发送的数据段都有序列号，接收方按序提交给应用层。如果收到乱序包，会暂存但重复发送上一个确认号的 ACK，触发快重传。

3）发送方启动定时器等待 ACK，一旦超时就重发。同时通过滑动窗口做流量控制，接收方通告自己的接收能力，防止缓冲区溢出。

4）拥塞控制动态调整发送速率。慢启动阶段指数增长，到阈值后线性增长。一旦丢包，认为网络拥塞，cwnd 直接减半进入拥塞避免。

代码上看，应用层调用 `socket.send()` 后，内核负责分段、加 TCP 头、维护发送队列。只有收到 ACK 才删除对应数据。丢包重试对应用透明。



整个过程里，序列号、确认机制和重传策略是**可靠传输**的三大支柱。没有这些，像 HTTP、MySQL 这类依赖稳定连接的应用根本搞不定。

Cookie、Session、Token 之间有什么区别？

这三个东西说白了都是为了解决 HTTP 无状态的问题，但思路完全不同。

Cookie 是浏览器端的一小块文本，服务器通过 Set-Cookie 响应头下发，浏览器自动带上 Cookie 请求头回传。它就存在客户端，能被 JS 读写（除非 HttpOnly），所以有 XSS 风险。一般用来存 session id 或用户偏好。

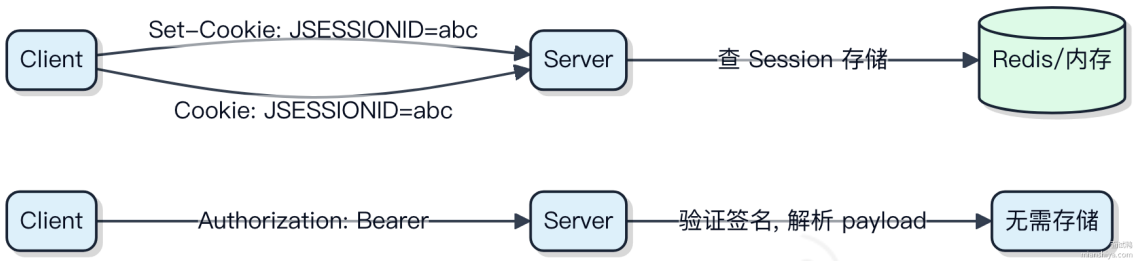
Session 是服务器端的存储机制。用户登录后，服务端生成一个 session 对象，用唯一 sessionId 当 key 存在内存或 Redis 里。sessionId 通常通过 Cookie 传给浏览器。下次请求带着这个 id，服务端就能查到状态。它的状态是集中管理的，登出可以直接销毁，但扩容麻烦，得搞 session 共享。

Token，尤其是 JWT，是无状态的方案。用户登录成功，服务器签发一个 Token，里面包含了用户信息和签名。客户端自己存，每次请求放 Header 里。服务器不存任何状态，每次靠验签解析数据。扩展性好，适合分布式，但 Token 一旦签发没法主动作废，除非加黑名单或者控制有效期很短。

1）Cookie 存在客户端，可以持久化 2）Session 存在服务端，依赖 Cookie 传 id 3）Token 完全自包含，服务端无状态

JWT 长这样：`header.payload.signature`，用点分割三段 Base64

要不要共享 Session，决定了你选哪种。单体系统用 Session 最省事，微服务基本都上 Token。



一台服务器，最多能建立多少 TCP 连接？

一台服务器能建立的 TCP 连接数，压根不是看“理论最大值”那么简单。表面上，TCP 用四元组（源IP、源端口、目标IP、目标端口）标识一个连接，如果只算客户端端口范围 65535，很多人觉得最多就几万连接。但这是典型误区。

实际上，服务器作为服务端，**源IP 和 源端口** 是由客户端决定的。只要客户端 IP 不同，或者同一个客户端用不同端口发请求，四元组就唯一，就能建立新连接。所以理论上，并发连接数可以远超 65535。

真正在生产环境卡脖子的是资源限制：

1）文件描述符：每个 TCP 连接在 Linux 上对应一个 socket 文件，受进程和系统级 fd 限制。默认单进程可能只有 1024，得通过 `ulimit` 和 `/etc/security/limits.conf` 改到几万甚至几十万。

2）内存：每个连接至少占用几 KB 内存（socket 结构、缓冲区等）。假设每个连接占 4KB，10 万连接就是 400MB，百万连接就得 4GB，内存得跟上。

3）端口复用：服务器可以通过 `SO_REUSEPORT` 或 `SO_REUSEADDR` 让多个进程监听同一端口，提升负载能力，Nginx、Netty 都这么干。

代码上，像 Netty 这种 NIO 框架，用少量线程就能管理海量连接，靠的是 epoll 多路复用，不会为每个连接开线程。

```java
// Netty 核心配置，支持高并发连接
EventLoopGroup boss = new NioEventLoopGroup(1);
EventLoopGroup worker = new NioEventLoopGroup();
ServerBootstrap b = new ServerBootstrap();
b.group(boss, worker).channel(NioServerSocketChannel.class)
 .childHandler(new ChannelInitializer<SocketChannel>() {
     public void initChannel(SocketChannel ch) {
         ch.pipeline().addLast(new MyHandler());
     }
 });
```

最终能扛住多少连接，得看系统调优程度和业务逻辑复杂度。C10K 问题早解决了，现在 C1M（百万连接）在合理架构下也能做到，比如阿里云 SLB、腾讯云 CLB 都是实打实跑着百万级长连接的。

OSI 七层模型是什么？

OSI 七层模型是网络通信的参考框架，把数据从一台设备传到另一台的过程拆成了七个层次，每一层各司其职，下层为上层服务。

1）物理层负责最底层的信号传输，比如网线、光纤这些介质上传输的高低电平，就是0和1的物理表示。
2）数据链路层管的是相邻设备之间的数据帧传输，像以太网（Ethernet）就在这层，交换机也是工作在这里，通过MAC地址寻址。
3）网络层负责跨网络的路径选择，IP协议就在这一层，路由器根据IP地址转发数据包。
4）传输层保证端到端的数据可靠或高效传输，TCP和UDP是典型代表。TCP提供连接管理、重传、流量控制，而UDP不搞这些，直接发。
5）会话层控制会话的建立、管理和终止，比如RPC调用或者数据库连接的会话保持。
6）表示层处理数据格式转换，比如加密解密、压缩解压、字符编码转换，让不同系统的数据能互相理解。
7）应用层直接面向用户程序，HTTP、FTP、DNS这些协议都在这层，你写的Web服务基本都跑在这一层之上。

实际互联网大多用的是TCP/IP四层模型，把前四层合并成“网络接口层”，会话层和表示层也常被并入应用层，但OSI模型在教学和标准化中还是很有指导意义。

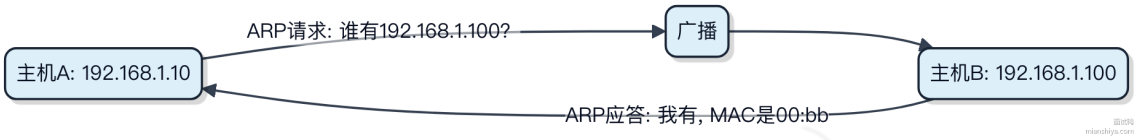
ARP 和 RARP 分别是什么？有什么区别？

ARP 和 RARP 都是 TCP/IP 协议栈里的地址解析协议，解决的是不同地址类型之间的映射问题。

ARP 是“地址解析协议”，用来根据 IP 地址查对应的 MAC 地址。比如一台主机要发数据包到局域网内的另一台机器，只知道对方的 IP，但以太网通信得靠 MAC 地址，这时候就得发 ARP 请求广播问：“谁有 192.168.1.100？” 对应的主机会回一个 ARP 应答，把自己的 MAC 告诉你。这个映射关系会缓存起来，一般存活几分钟。

RARP 则反过来，是“反向地址解析协议”，根据 MAC 地址来获取 IP 地址。这在早期无盘工作站上用得多，设备自己没有存储，开机时只知道自己的 MAC，需要通过 RARP 服务器告诉它该用哪个 IP。不过 RARP 功能太简单，已经被 DHCP 取代了，现在基本见不到了。

区别主要在方向和用途： 1） ARP：IP → MAC，现在仍然广泛使用，尤其是局域网通信。 2） RARP：MAC → IP，已被 DHCP 淘汰，属于历史协议。



简单说，ARP 是网络里找“这个 IP 在哪”的常用手段，而 RARP 是老设备“我是谁”的求助方式，早就不怎么用了。

TCP 滑动窗口的作用是什么？

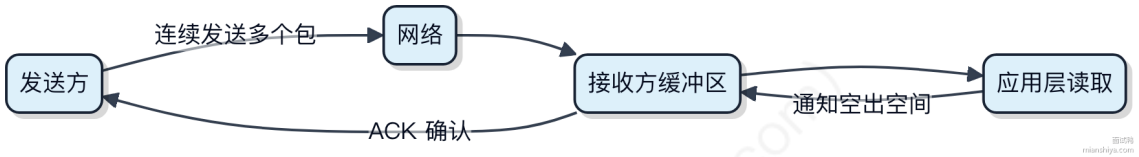
滑动窗口的核心是解决可靠传输下的效率问题。TCP 要保证不丢包、不乱序，但如果每发一个包就等 ACK 再发下一个，延迟会把吞吐压垮。滑动窗口就是让发送方不用等确认，可以连续把多个数据包扔出去，相当于“批量发货”。

接收方有个缓冲区，能存还没被应用读走的数据。这个缓冲区大小就是接收窗口（rwnd），它告诉发送方自己还能收多少。发送方根据这个值维护自己的发送窗口，已发未确认的包不能删，但可以继续发新包，直到窗口填满。

1）窗口左边界是最早发出去还没确认的包
2）右边界是还能发的最大序号
3）收到 ACK 后左边界右移，窗口“滑动”

如果接收方处理慢，缓冲区满了，就会把窗口设为 0，发送方暂停。等应用读走数据空出空间，接收方再发个“窗口更新”报文恢复传输。

网络拥塞时，光看接收方窗口不够，还得考虑链路承载能力。所以有拥塞窗口（cwnd），和 rwnd 取小值作为实际可用窗口。慢启动、拥塞避免这些机制就是在动态调整 cwnd。



TCP 的 SACK 的引入是为了解决什么问题？

TCP 的 SACK（选择性确认）主要是为了解决 **丢包严重时，传统累计确认机制效率低下**的问题。

没有 SACK 的时候，接收方只能告诉发送方“我收到了哪些连续的数据”，比如收到 1、2、3 和 5、6，但 4 没到，那它只能说“我收到前 3 个”。发送方不知道后面那些非连续的包其实已经到了，只能认为从第 4 个开始全丢了，于是把 4、5、6 都重传一遍。这种情况下，明明 5、6 已经收过了，却还要再收一次，浪费带宽，延迟也上去了。

SACK 允许接收方在 ACK 包里额外带上“我还收到了哪些不连续的块”，比如“3 到了，但我还收到了 5-6 这一段”。发送方一看就知道，只有 4 真丢了，5 和 6 不用重传。这样就避免了重复传输，尤其在网络差、多包乱序的场景下，像跨公网传输或者移动网络环境，提升非常明显。

像 Linux 内核默认就开启了 SACK，很多高性能传输场景比如文件服务器、CDN 回源都依赖它来扛住高丢包率下的吞吐。

注意，SACK 是可选项，得双方都支持才行。有些中间设备比如防火墙会把它过滤掉，导致性能回退到老模式。

TCP 有超时重传为什么还需要快速重传机制？

超时重传靠 RTO 定时器，等的时间比较长，一般几百毫秒起步。如果只依赖它，一旦丢包，整个连接就得干等，性能直接垮掉。

其实 TCP 收到乱序数据包时会触发重复 ACK。比如接收方期望收 seq=1001，但来了 2001，它就会一直回 ack=1001。发送方连续收到 3 个相同 ack，就意识到这个包可能丢了，马上重传，不用等到超时。

这就是快速重传的核心：用重复 ACK 当信号，实现**更快的丢包检测**。像下载大文件、视频流这种场景，中间丢一个包，靠快速重传能在几十毫秒内恢复，体验好太多了。

不过它也有局限，比如低速链路可能凑不够 3 个重复 ACK，这时候还得 fallback 到超时重传兜底。

所以两种机制是互补的。超时重传保底不丢，快速重传提效。现代 TCP 栈都是两者结合，才能既可靠又高效。

为什么 TCP 挥手需要有 TIME_WAIT 状态？

TCP 四次挥手中，主动关闭连接的一方会进入 TIME_WAIT 状态，这个状态的存在不是设计缺陷，而是为了解决两个关键问题。

1）确保最后一个 ACK 能被对方收到。如果这个 ACK 丢了，被动关闭方会重传 FIN，此时如果主动方已经彻底关闭，就无法响应，导致对方不能正常释放连接。留一个 TIME_WAIT 时间（一般是 2MSL），就是为了能重发这个 ACK。

2）防止旧连接的残留数据包干扰新连接。TCP 协议用四元组（源IP、源端口、目的IP、目的端口）标识一个连接。如果连接快速复用端口，网络中延迟的老数据包可能会被误认为是新连接的数据。等待 2MSL 能让所有老数据在网络中自然消失。

MSL 是报文最大生存时间，一般取 30 秒到 2 分钟不等，所以 TIME_WAIT 状态最长可能持续 4 分钟。虽然它保障了可靠性，但在高并发短连接场景下，比如 Web 服务器大量请求，容易出现端口耗尽的问题。这时候可以通过开启 `SO_REUSEADDR` 选项，允许处于 TIME_WAIT 的 socket 被复用，缓解这个问题。

对于客户端来说，频繁建立短连接要特别注意 TIME_WAIT 的累积；服务端通常不会成为主动关闭方，受影响较小。Nginx、Tomcat 这类服务在调优时都会关注这个状态的数量。

什么是 SYN Flood 攻击？

SYN Flood 是一种经典的 DDoS 攻击手段，目标是耗尽服务器的连接资源，让正常请求没法建立 TCP 连接。

TCP 建连要三次握手。攻击者发一个 SYN 包，服务器收到后进入 `SYN_RECEIVED` 状态，分配一块内存保存这个半连接，并回 SYN+ACK。正常情况下客户端会回 ACK，连接就建立了。但 SYN Flood 的关键在于——**压根不回 ACK**。

这时候服务器那边得等着，连接还占着内存和端口资源。Linux 内核默认会重试 5 次（`tcp_synack_retries=5`），每次等 30 秒左右，一个连接能卡住将近 2 分钟。成千上万的伪造源 IP 的 SYN 包打过来，半连接队列（`syn queue`）很快被打满，新的请求进不来，服务就挂了。

常见防护手段有几种：1）开启 **SYN Cookies**，这是最常用的方案。它不立即分配资源，而是把连接信息编码到返回的 ACK 序号里，等到对方回 ACK 时再校验并重建连接。这样即使队列满了也能抗住。2）缩短 `tcp_synack_retries`，加快释放速度。3）加大 `somaxconn` 和 `backlog`，提升队列容量。4）用防火墙或中间件如 **F5、Cloudflare** 过滤异常流量。

其实现在很多云厂商的负载均衡已经内置了这类防御机制，但理解原理对排查网络故障和设计高可用架构依然很重要。

TCP 三次握手时，发送 SYN 之后就宕机了会怎么样？

客户端发完 SYN 后直接宕机，服务端就卡在 SYN_RCVD 状态，对应的半连接会留在 backlog 队列里。这个状态不会一直占着，**TCP 有定时重传机制**。

服务端发了 SYN-ACK 后收不到 ACK，就会按 tcp_synack_retries 的配置重试，默认一般是 5 次，每次间隔约 1 秒、2 秒、4 秒这样指数退避。大概等 30 到 60 秒后彻底放弃，释放这个半连接。

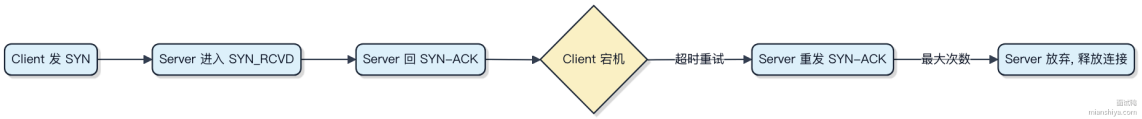
这段时间里，连接资源是被占用的，包括 socket 结构体、内存，还有 backlog 队列里的位置。如果这种半死不活的连接太多，比如遇到恶意 SYN Flood 攻击，队列满了，正常的连接请求就进不来，服务就搞不定。

所以生产环境得防这类问题。Linux 提供了几个机制：

1）开启 **syncookies**，这是最常用的手段。开启后，即使 backlog 满了，也能通过 cookie 的方式验证后续的 ACK，合法连接依然能建立，相当于绕过了队列限制。

2）调小 tcp_synack_retries 和 tcp_syn_timeout 缩短等待周期，快速释放无效资源。

3）加大 somaxconn 和 backlog 参数，提升瞬时抗压能力。



这种情况其实在云上挺常见的，网络抖动或者客户端异常退出都可能触发。关键是服务端要有应对半连接堆积的能力，否则很容易被拖垮。

TCP 的粘包和拆包能说说吗？

TCP 本身没有“包”的概念，它传输的是字节流。所谓粘包和拆包，其实是应用层读取数据时，没按预期边界处理导致的。

发送方连续发两条消息，接收方可能一次性收到两条（粘包），也可能一条被分成两次读到（拆包）。根本原因是 TCP 不保证收发消息的次数对等，只保证字节顺序正确。

常见场景比如用 Netty 做通信框架，如果没加 **编解码器** 处理消息边界，就会出问题。解决方案得在协议层面定好规则：

1）固定消息长度，不够补零
2）用特殊分隔符，比如换行符或特殊字符
3）头+体格式，头部存实际数据长度

最常用的是第三种，像 Dubbo、RocketMQ 内部都用 LengthFieldBasedFrameDecoder 来解。

举个例子，你发了两个 100 字节的消息，TCP 可能合并成一个 200 字节的包发过去，也可能因为 MTU 拆成 150 + 50。接收端必须靠协议解析才能还原原始消息边界。

不处理粘包拆包，服务间通信基本搞不定。尤其做 RPC 或网关，编解码这步压根不经过业务代码，全靠框架提前配好。

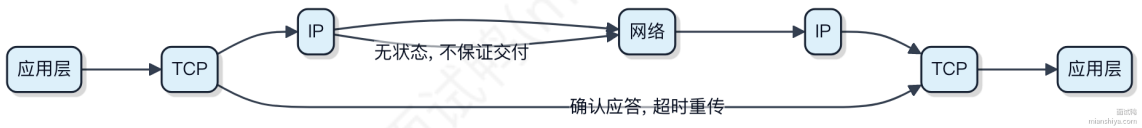
为什么要 TCP，IP 层实现控制不行么？

IP 层只负责把数据包从一个主机送到另一个，它不关心数据有没有丢、乱序或者损坏。你可以把它想象成邮局寄信，只管投递，不管收件人有没有收到、是不是顺序对。

TCP 的价值在于在不可靠的网络上构建了**可靠传输**。它要处理丢包重传、超时控制、流量控制、拥塞控制，这些 IP 层压根不经过，也搞不定。比如你用 HTTP 下载一个文件，中间丢了几个包，IP 层不会察觉，但 TCP 会通过确认机制发现并重新发送。

再看使用场景，像 MySQL 主从复制、Kafka 生产消费消息，都依赖连接的可靠性。如果只靠 IP，一旦网络抖动，数据就残缺了，整个系统状态就乱了。

代码上看，Socket 编程里 write 之后，操作系统会保证数据最终被对方 read 到，这背后就是 TCP 在做缓冲、重传、排序的脏活累活。



UDP 就是只用 IP 服务的典型，适合视频通话这种能容忍少量丢失但要求低延迟的场景。而需要数据完整的，基本都得往上走一层，靠 TCP 或基于 UDP 自行实现可靠逻辑，成本高得多。

HTTP 和 HTTPS 有什么区别？

HTTP 和 HTTPS 的本质区别在于**安全性**，HTTPS 不是新的协议，而是 HTTP 跑在 SSL/TLS 之上。

1）通信内容加密
HTTP 明文传输，中间人能直接看到请求和响应内容。HTTPS 在 TCP 握手后、HTTP 通信前，先通过 SSL/TLS 协议协商密钥，后续数据都用对称加密传输。比如浏览器访问支付宝，账号密码要是走 HTTP，网络里的路由器都能看见。

2）身份认证
HTTPS 需要证书来证明“你真的是你”。服务端把数字证书发给客户端，客户端通过 CA（比如 Let's Encrypt、DigiCert）验证证书合法性。没有这一步，钓鱼网站随便仿个淘宝页面就搞定了。

3）防篡改
TLS 会对数据做完整性校验，用 MAC（消息认证码）机制。哪怕中间有人改了一个字节，接收方解密时就会发现校验失败，直接断开。

性能上，HTTPS 多了握手过程，首次建立连接要多花 1-2 个 RTT。但现代 TLS 1.3 已优化到 1-RTT 甚至 0-RTT，加上 CDN 普遍支持，实际差距很小。主流站点像 Google、淘宝、微信支付全默认 HTTPS。

代码层面，Java 里用 `HttpsURLConnection` 替代 `HttpURLConnection`，关键是要信任的证书库。Netty 或 Spring WebFlux 中配置 SSLContext 即可启用。

现在没有理由再用纯 HTTP，搜索引擎也优先收录 HTTPS 站点。小项目可以用 Nginx + 免费证书快速上 HTTPS。

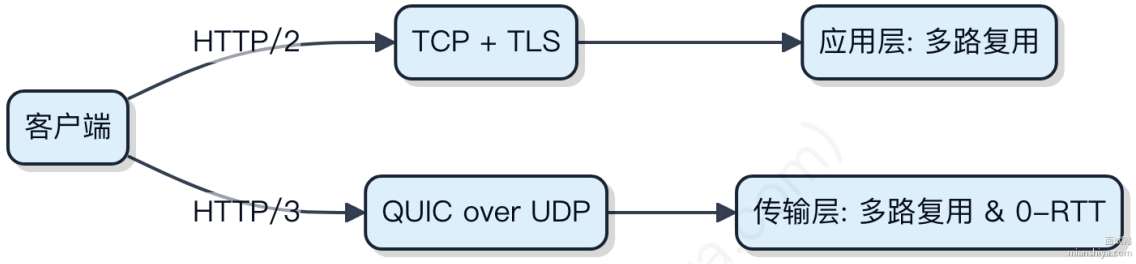
HTTP 2.0 和 3.0 有什么区别？

HTTP/2 的核心是多路复用，它把多个请求塞进同一个 TCP 连接里，并发传输。每个请求拆成帧，通过流（Stream）来区分，这样就避免了 HTTP/1.1 的队头阻塞问题。但底层还是 TCP，万一丢包，整个连接都得等重传，影响所有流。

HTTP/3 直接改了传输层，用 **QUIC** 协议替换了 TCP。QUIC 跑在 UDP 上，自己实现可靠传输和拥塞控制。最关键的是，它把连接绑定到客户端的连接 ID，而不是 IP + 端口。切换网络时，比如从 Wi-Fi 切 4G，连接不会断，直接续传。而且 QUIC 的多路复用是在传输层做的，丢一个流不影响其他流，彻底解决了队头阻塞。

实际应用中，Google 早就用 QUIC 撑起大部分流量，Cloudflare、Fastly 这些 CDN 厂商也全面支持 HTTP/3。国内阿里云、腾讯云也在逐步铺开。

代码层面你看不到太多变化，因为 HTTP/3 是浏览器和服务器自动协商的。但服务端配置要注意，比如 Nginx 需要编译 BoringSSL 或 quiche 支持，监听 UDP 443 端口。



简单说，HTTP/2 解决了应用层的并发问题，HTTP/3 连协议切换、移动网络切换这些场景都扛住了，更适应现代网络环境。

HTTP 1.0 和 2.0 有什么区别？

HTTP/2 最大的变化是引入了**二进制分帧层**，这直接改掉了 HTTP/1.1 的“一请求一响应”、串行传输的老毛病。以前每个请求都要单独建 TCP 连接，或者走管道化但容易阻塞，现在一个连接就能并发处理多个请求，效率提升非常明显。

1）头部压缩。HTTP/1.1 每次都传完整头，重复的 Cookie、User-Agent 这些字段特别浪费。HTTP/2 用 HPACK 算法压缩头部，相同字段只传差量，能省下 30% 到 50% 的头部流量。

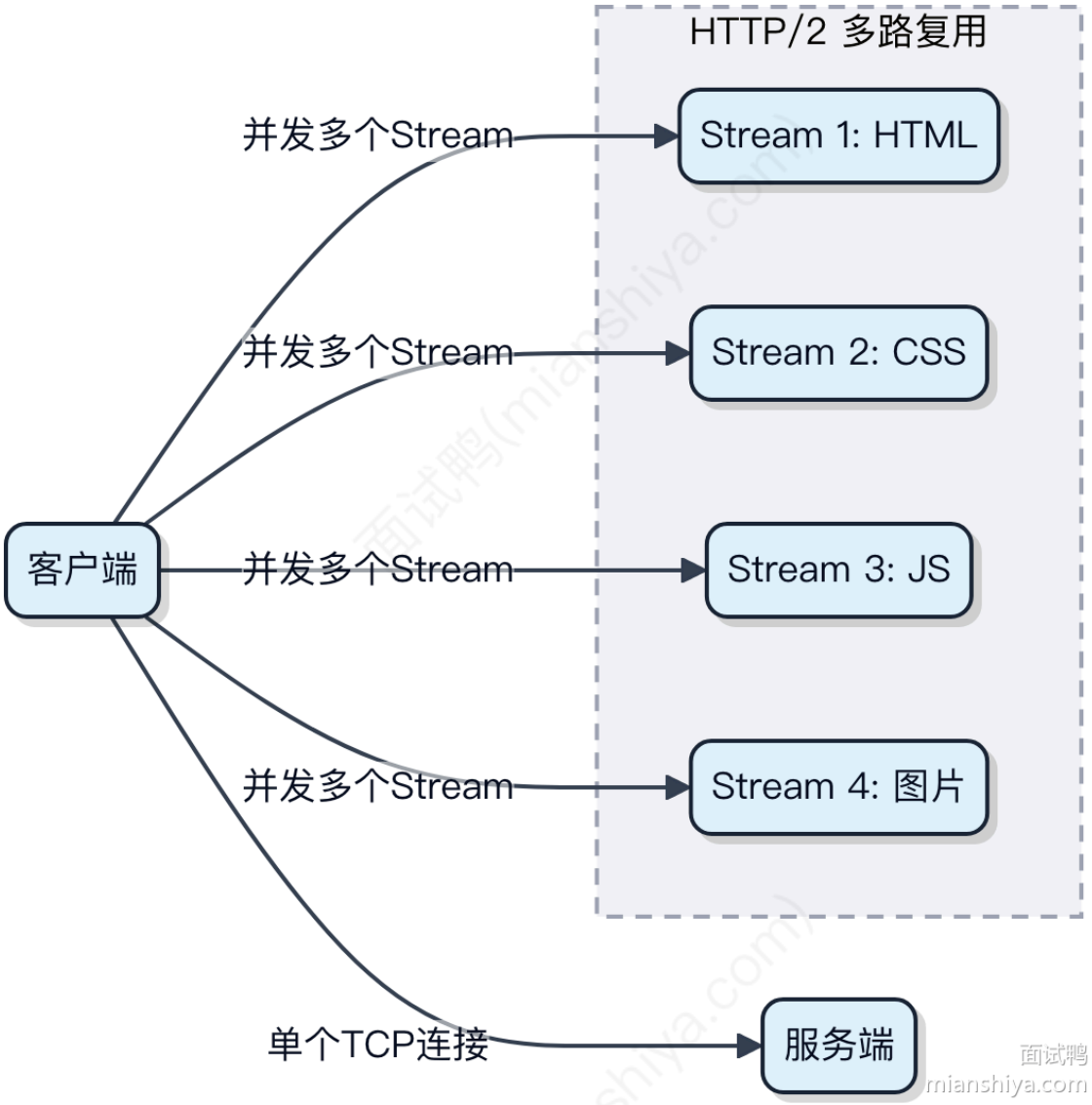
2）多路复用。这是最核心的体验提升。HTTP/1.1 队头阻塞的问题在 HTTP/2 里被解决了，多个请求和响应可以同时在一个 TCP 连接上通过独立的数据帧交错传输，谁也不等谁。

3）服务器推送。这个功能允许服务端提前把客户端可能需要的资源推过去，比如页面要加载 main.css，服务端可以直接推，不用等浏览器解析 HTML 再发请求。不过实际用得不多，很多场景反而会增加负担。

4）优先级机制。客户端可以给不同资源设置优先级，比如 JS、CSS 比图片更重要，服务端就可以按序发送，优化页面渲染体验。

虽然有这些优势，但 HTTP/2 对网络环境要求更高。丢包严重时，因为所有数据都在一个 TCP 流里，一旦出错整个连接都会卡住。这也是后来 HTTP/3 改用 QUIC 协议的原因之一。

面试鸭(mianshiya.com)



从网络角度来看，用户从输入网址到网页显示，期间发生了什么？

用户敲下回车后，浏览器要先拿到服务器 IP 地址。本地查 hosts 文件，没找到就发 DNS 查询请求。这个请求通常先到本地 DNS 服务器，它会递归或迭代查询根域名、顶级域、权威域名服务器，最终拿到 A 记录。

有了 IP，浏览器发起 TCP 连接，三次握手建立连接。如果走 HTTPS，接着进行 TLS 握手，协商加密套件、交换密钥，确保后续通信安全。

TCP 连接建立后，浏览器开始发 HTTP 请求。比如 GET /index.html，携带 Host、User-Agent 等头信息。请求通过操作系统网络栈，经过路由器层层转发，最终到达目标服务器。

服务器收到请求，应用层如 Nginx 或 Tomcat 处理，可能访问数据库、缓存，生成响应内容。响应包含状态码、响应头和 HTML 内容，通过已建立的 TCP 连接返回。

浏览器收到响应，解析 HTML，遇到 CSS、JS、图片等资源，再发起多个并行请求。这些请求可能复用现有 TCP 连接（HTTP Keep-Alive），减少握手开销。

页面渲染同时进行，构建 DOM 树、CSSOM，生成渲染树，布局绘制。JS 执行可能阻塞解析，所以一般建议异步加载。

整个过程涉及多层协议协作。DNS 查询可能耗时几百毫秒，CDN 和 DNS 缓存能显著降低延迟。现代浏览器对同域名并发连接数限制在 6 个左右，超出的请求排队。

整个链路中任意环节超时或失败都会导致页面加载异常，比如 DNS 污染、TCP 重传、TLS 协商失败、后端服务无响应等。

除了四次挥手，还有什么方法断开连接？

连接断开不只有四次挥手这一种方式，尤其在异常或极端场景下，系统有很多“非常规”手段。

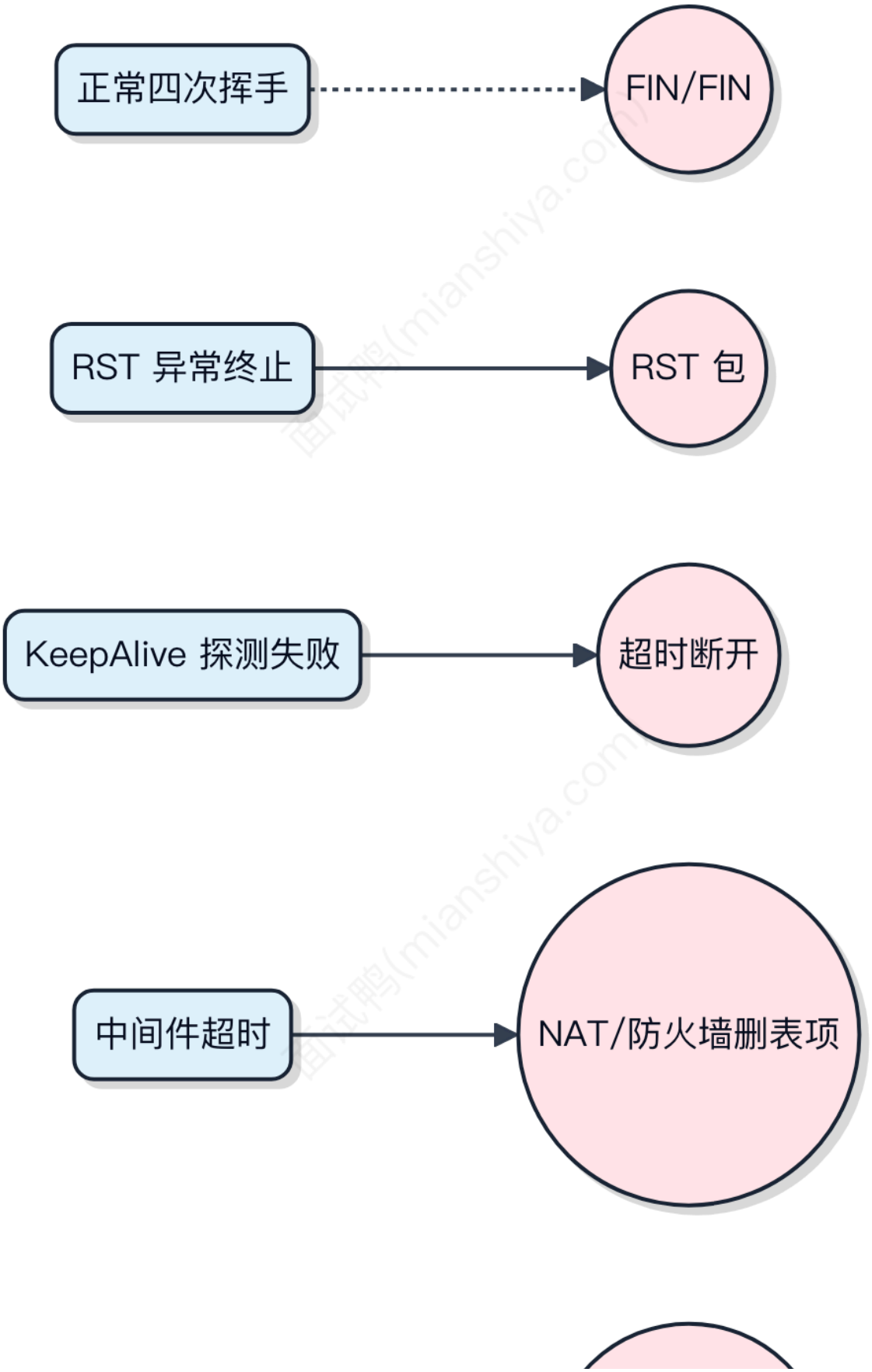
1）RST 报文强制终止
当一方收到一个非法的报文段（比如发给一个已经关闭的端口），内核会直接回一个 RST 标志位的包。对方收到 RST 后立刻释放连接，**不再走任何挥手流程**。像 nginx 配置了 `reset_timedout_connection on` 时，超时后就可能发 RST 强制断开。

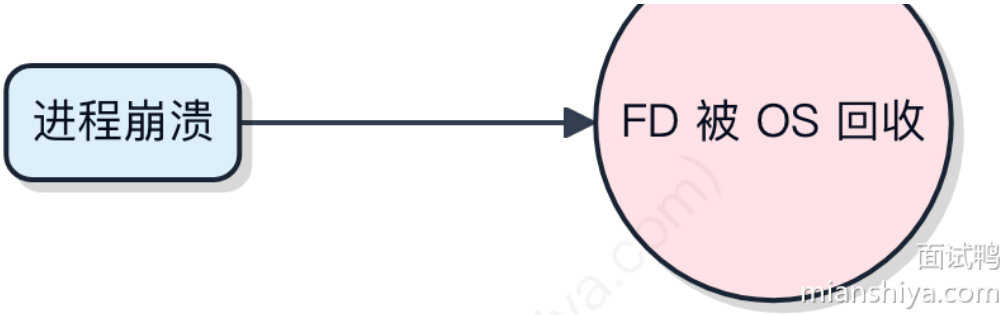
2）连接保活机制探测失败
TCP 自带 keepalive 机制，开启后如果连续多次探测无响应（默认 75 秒探测，9 次失败），就认为对端挂了，直接断开。Netty、Dubbo 这类框架通常自己实现应用层心跳，检测到异常就主动 close。

3）资源耗尽或进程崩溃
进程崩溃操作系统会回收其所有 socket，内核会批量释放连接。类似 OOM killer 干掉 Java 进程，所有连接自然消失。这时候对方如果没有读写，可能很久才发现连接断了。

4）防火墙/NAT 中间设备干预
云环境里 NAT 网关或安全组可能维护连接表，超时（比如 900 秒）自动清理。一旦记录没了，后续包被丢弃，连接实际已断，但两端都未必立刻知道。

面试鸭(mianshiya.com)


正常四次挥手
FIN/FIN
RST 异常终止
RST 包
KeepAlive 探测失败
超时断开
中间件超时
NAT/防火墙删表项



RST 和中间件超时是线上问题高发区，抓包看有没有 **RST 标志位**或结合 netstat 观察连接状态变化能快速定位。

TCP 初始序列号 ISN 怎么取值的？

TCP 连接建立时，初始序列号 ISN 不是随便给个 0 或者 1 开始的，它是有算法依据的，主要为了防止历史重复数据被误当成新连接的数据包。

ISN 的取值核心是一个**伪随机递增的时钟计数器**。RFC 793 规定，ISN 每 4 微秒加 1，这样每过大约 4.55 小时就会循环一次。这个设计很关键，能避免短时间内重用相同的序列号。

实际实现中，比如 Linux 内核，并不会真的从固定起点开始每 4 微秒加一，而是结合系统启动时间、源/目的 IP 和端口等信息做哈希扰动，生成一个**初始偏移量**，再在这个基础上按时间递增。这么做是为了防止被预测，提升安全性。

1）连接发起方在 SYN 包里带上自己的 ISN
2）接收方记录这个值，后续确认和数据传输都基于此偏移计算
3）对方回 SYN+ACK 时也带自己的 ISN，双方各自维护独立的序列号空间

如果两个方向用相同起始值，旧连接的延迟报文可能被新连接错误接收。所以双向 ISN 必须独立生成。

总之，ISN 是靠**时间戳 + 随机扰动**生成的，既保证唯一性又防猜测，这是 TCP 可靠传输的基础之一。

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*